

```
1 <!doctype html>
2 <html lang="zh-CN">
3 <head>
4   <meta charset="utf-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <title>双线性 Hall 高精度位置检测与补偿软件</title>
7   <style>
8     :root {
9       --bg: #f5fbff;
10      --panel: #ffffff;
11      --panel-2: #edf7ff;
12      --line: #d6e9f7;
13      --text: #19384d;
14      --muted: #60788a;
15      --blue: #57a9df;
16      --blue-dark: #1672ad;
17      --blue-soft: #dff2ff;
18      --green: #2ba86a;
19      --red: #d64545;
20      --orange: #d9812b;
21      --shadow: 0 18px 42px rgba(38, 118, 166, 0.12);
22      --radius: 8px;
23    }
24
25    * { box-sizing: border-box; }
26
27    body {
28      margin: 0;
29      font-family: "Microsoft YaHei", "Segoe UI", Arial, sans-serif;
30      background:
31        linear-gradient(180deg, rgba(230, 246, 255, 0.95), rgba(248, 252, 255, 0.95)),
32        radial-gradient(circle at 20% 0%, rgba(111, 189, 235, 0.18), transparent 34%);
33      color: var(--text);
34    }
35
36    .app-shell {
37      min-height: 100vh;
38      display: grid;
39      grid-template-rows: auto 1fr;
40    }
41
42    header {
43      position: sticky;
44      top: 0;
45      z-index: 10;
46      display: grid;
47      grid-template-columns: 1fr auto;
48      gap: 18px;
49      align-items: center;
50      padding: 14px 22px;
51      background: rgba(255, 255, 255, 0.92);
52      border-bottom: 1px solid var(--line);
53      backdrop-filter: blur(12px);
```

```
54     }
55
56     .brand {
57         display: flex;
58         align-items: center;
59         gap: 12px;
60         min-width: 0;
61     }
62
63     .brand-mark {
64         width: 38px;
65         height: 38px;
66         display: grid;
67         place-items: center;
68         border-radius: 50%;
69         background: linear-gradient(135deg, #dff2ff, #8fd0f5);
70         color: var(--blue-dark);
71         font-weight: 800;
72         border: 1px solid #9dd8f7;
73         flex: none;
74     }
75
76     h1 {
77         margin: 0;
78         font-size: 20px;
79         letter-spacing: 0;
80         line-height: 1.35;
81         white-space: nowrap;
82         overflow: hidden;
83         text-overflow: ellipsis;
84     }
85
86     .subtitle {
87         margin-top: 2px;
88         color: var(--muted);
89         font-size: 12px;
90     }
91
92     .toolbar {
93         display: flex;
94         align-items: center;
95         gap: 8px;
96         flex-wrap: wrap;
97         justify-content: flex-end;
98     }
99
100    button {
101        border: 1px solid var(--line);
102        background: var(--panel);
103        color: var(--text);
104        padding: 8px 12px;
105        border-radius: 6px;
106        font: inherit;
```

```
107     cursor: pointer;
108     transition: 0.16s ease;
109     white-space: nowrap;
110 }
111
112 button:hover {
113     border-color: #9bd4f3;
114     background: #f2faff;
115     transform: translateY(-1px);
116 }
117
118 button.primary {
119     background: var(--blue-dark);
120     color: #fff;
121     border-color: var(--blue-dark);
122     box-shadow: 0 8px 22px rgba(22, 114, 173, 0.22);
123 }
124
125 button.primary:hover {
126     background: #0f649b;
127 }
128
129 main {
130     display: grid;
131     grid-template-columns: minmax(310px, 360px) 1fr;
132     gap: 16px;
133     padding: 16px;
134 }
135
136 aside {
137     min-width: 0;
138 }
139
140 .panel {
141     background: var(--panel);
142     border: 1px solid var(--line);
143     border-radius: var(--radius);
144     box-shadow: var(--shadow);
145 }
146
147 .control-panel {
148     padding: 14px;
149     display: grid;
150     gap: 14px;
151     max-height: calc(100vh - 86px);
152     overflow: auto;
153 }
154
155 .section {
156     border-top: 1px solid var(--line);
157     padding-top: 12px;
158     display: grid;
159     gap: 10px;
```

```
160     }
161
162     .section:first-child {
163         border-top: 0;
164         padding-top: 0;
165     }
166
167     .section h2 {
168         margin: 0;
169         font-size: 15px;
170         display: flex;
171         justify-content: space-between;
172         align-items: center;
173     }
174
175     .tag {
176         display: inline-flex;
177         align-items: center;
178         height: 22px;
179         padding: 0 8px;
180         border-radius: 999px;
181         background: var(--blue-soft);
182         color: var(--blue-dark);
183         font-size: 12px;
184         font-weight: 600;
185     }
186
187     .field-grid {
188         display: grid;
189         grid-template-columns: 1fr 1fr;
190         gap: 9px;
191     }
192
193     label.field {
194         display: grid;
195         gap: 5px;
196         color: var(--muted);
197         font-size: 12px;
198         min-width: 0;
199     }
200
201     input, select {
202         width: 100%;
203         border: 1px solid #c9e2f2;
204         border-radius: 6px;
205         padding: 7px 8px;
206         background: #fbfdff;
207         color: var(--text);
208         font: inherit;
209         min-height: 34px;
210     }
211
212     input:focus, select:focus {
```

```
213     outline: 2px solid rgba(87, 169, 223, 0.22);
214     border-color: var(--blue);
215 }
216
217 .check-list {
218     display: grid;
219     grid-template-columns: 1fr 1fr;
220     gap: 8px;
221 }
222
223 .check-item {
224     display: flex;
225     align-items: center;
226     gap: 7px;
227     min-height: 32px;
228     padding: 5px 8px;
229     border: 1px solid var(--line);
230     border-radius: 6px;
231     background: #f9fdff;
232     font-size: 13px;
233 }
234
235 .check-item input {
236     width: 16px;
237     height: 16px;
238     min-height: 0;
239 }
240
241 .range-line {
242     display: grid;
243     grid-template-columns: 1fr 56px;
244     gap: 8px;
245     align-items: center;
246 }
247
248 .range-line.wide {
249     grid-template-columns: 1fr 74px;
250 }
251
252 input[type="range"] {
253     padding: 0;
254     accent-color: var(--blue-dark);
255 }
256
257 .workspace {
258     min-width: 0;
259     display: grid;
260     grid-template-rows: auto auto 1fr;
261     gap: 14px;
262 }
263
264 .summary-strip {
265     display: grid;
```

```
266     grid-template-columns: repeat(4, minmax(0, 1fr));
267     gap: 10px;
268 }
269
270 .metric {
271     padding: 13px;
272     background: var(--panel);
273     border: 1px solid var(--line);
274     border-radius: var(--radius);
275     box-shadow: var(--shadow);
276     min-width: 0;
277 }
278
279 .metric .name {
280     color: var(--muted);
281     font-size: 12px;
282 }
283
284 .metric .value {
285     margin-top: 5px;
286     font-size: 22px;
287     font-weight: 750;
288     color: var(--blue-dark);
289     white-space: nowrap;
290     overflow: hidden;
291     text-overflow: ellipsis;
292 }
293
294 .metric .note {
295     margin-top: 3px;
296     color: var(--muted);
297     font-size: 12px;
298 }
299
300 .tabs {
301     display: flex;
302     gap: 8px;
303     padding: 6px;
304     background: rgba(255, 255, 255, 0.78);
305     border: 1px solid var(--line);
306     border-radius: var(--radius);
307     width: fit-content;
308     max-width: 100%;
309     overflow: auto;
310 }
311
312 .tab {
313     border: 0;
314     background: transparent;
315     padding: 8px 12px;
316     border-radius: 5px;
317     color: var(--muted);
318     box-shadow: none;
```

```
319     }
320
321     .tab.active {
322         background: var(--blue-dark);
323         color: #fff;
324     }
325
326     .view {
327         display: none;
328         min-width: 0;
329     }
330
331     .view.active {
332         display: block;
333     }
334
335     .chart-grid {
336         display: grid;
337         grid-template-columns: repeat(2, minmax(0, 1fr));
338         gap: 14px;
339     }
340
341     .chart-card.wide {
342         grid-column: 1 / -1;
343     }
344
345     .principle-grid {
346         display: grid;
347         grid-template-columns: repeat(2, minmax(0, 1fr));
348         gap: 14px;
349     }
350
351     .principle-card {
352         padding: 14px;
353         background: var(--panel);
354         border: 1px solid var(--line);
355         border-radius: var(--radius);
356         box-shadow: var(--shadow);
357         min-width: 0;
358     }
359
360     .principle-card h3 {
361         margin: 0 0 8px;
362         color: var(--blue-dark);
363         font-size: 15px;
364     }
365
366     .principle-card p {
367         margin: 0 0 8px;
368         color: var(--muted);
369         line-height: 1.65;
370         font-size: 13px;
371     }
```

```
372
373 .formula {
374     margin: 8px 0;
375     padding: 10px 12px;
376     background: #f4fbff;
377     border: 1px solid #d7ecf8;
378     border-left: 4px solid var(--blue);
379     border-radius: 6px;
380     color: #19384d;
381     font-family: "Cambria Math", "Times New Roman", serif;
382     font-size: 15px;
383     line-height: 1.55;
384     overflow-x: auto;
385 }
386
387 .mini-list {
388     margin: 8px 0 0;
389     padding-left: 18px;
390     color: var(--muted);
391     font-size: 13px;
392     line-height: 1.6;
393 }
394
395 .chart-card {
396     background: var(--panel);
397     border: 1px solid var(--line);
398     border-radius: var(--radius);
399     box-shadow: var(--shadow);
400     padding: 12px;
401     min-width: 0;
402 }
403
404 .chart-head {
405     display: flex;
406     justify-content: space-between;
407     align-items: center;
408     gap: 10px;
409     margin-bottom: 8px;
410 }
411
412 .chart-title {
413     font-weight: 700;
414     font-size: 14px;
415 }
416
417 .legend {
418     display: flex;
419     flex-wrap: wrap;
420     gap: 8px 12px;
421     font-size: 12px;
422     color: var(--muted);
423 }
424
```



```
425 .legend span {
426     display: inline-flex;
427     align-items: center;
428     gap: 5px;
429 }
430
431 .swatch {
432     width: 18px;
433     height: 3px;
434     border-radius: 3px;
435     background: currentColor;
436 }
437
438 canvas {
439     display: block;
440     width: 100%;
441     height: 280px;
442     background: #fff;
443     border: 1px solid #e2eef6;
444     border-radius: 6px;
445 }
446
447 canvas.model-canvas {
448     height: 340px;
449     background: linear-gradient(180deg, #fafdff 0%, #edf8ff 100%);
450 }
451
452 .metrics-viz-grid {
453     display: grid;
454     grid-template-columns: 1.15fr 0.85fr;
455     gap: 14px;
456     margin-bottom: 14px;
457 }
458
459 .table-wrap {
460     overflow: auto;
461     background: var(--panel);
462     border: 1px solid var(--line);
463     border-radius: var(--radius);
464     box-shadow: var(--shadow);
465 }
466
467 table {
468     border-collapse: collapse;
469     width: 100%;
470     min-width: 720px;
471     background: #fff;
472 }
473
474 th, td {
475     padding: 10px 12px;
476     border-bottom: 1px solid #e6f1f8;
477     text-align: left;
```

```
478     font-size: 13px;
479   }
480
481   th {
482     background: #e8f6ff;
483     color: var(--blue-dark);
484     font-weight: 750;
485     position: sticky;
486     top: 0;
487   }
488
489   tr:last-child td {
490     border-bottom: 0;
491   }
492
493   .algo-flow {
494     display: grid;
495     grid-template-columns: repeat(6, minmax(110px, 1fr));
496     gap: 10px;
497     align-items: stretch;
498   }
499
500   .flow-step {
501     position: relative;
502     min-height: 76px;
503     padding: 12px;
504     background: #ffffff;
505     border: 1px solid #bde0f5;
506     border-radius: var(--radius);
507     box-shadow: var(--shadow);
508   }
509
510   .flow-step::after {
511     content: "";
512     position: absolute;
513     right: -10px;
514     top: 50%;
515     width: 10px;
516     height: 2px;
517     background: #9ccfe8;
518   }
519
520   .flow-step:last-child::after {
521     display: none;
522   }
523
524   .step-no {
525     width: 24px;
526     height: 24px;
527     border-radius: 50%;
528     display: grid;
529     place-items: center;
530     background: var(--blue-dark);
```

```
531     color: #fff;
532     font-size: 12px;
533     font-weight: 700;
534     margin-bottom: 8px;
535 }
536
537 .step-name {
538     font-weight: 700;
539     font-size: 13px;
540 }
541
542 .step-note {
543     color: var(--muted);
544     font-size: 12px;
545     margin-top: 4px;
546     line-height: 1.4;
547 }
548
549 .report-grid {
550     display: grid;
551     grid-template-columns: 1.1fr 0.9fr;
552     gap: 14px;
553 }
554
555 .report-box {
556     padding: 14px;
557     background: var(--panel);
558     border: 1px solid var(--line);
559     border-radius: var(--radius);
560     box-shadow: var(--shadow);
561 }
562
563 .report-box h3 {
564     margin: 0 0 10px;
565     font-size: 15px;
566 }
567
568 .report-box p {
569     margin: 0 0 9px;
570     color: var(--muted);
571     line-height: 1.7;
572     font-size: 13px;
573 }
574
575 .toast {
576     position: fixed;
577     right: 18px;
578     bottom: 18px;
579     z-index: 20;
580     padding: 12px 14px;
581     border-radius: 6px;
582     background: #12384f;
583     color: #fff;
```

```

584     box-shadow: 0 12px 30px rgba(0, 0, 0, 0.22);
585     opacity: 0;
586     transform: translateY(12px);
587     pointer-events: none;
588     transition: 0.2s ease;
589     max-width: min(420px, calc(100vw - 36px));
590 }
591
592 .toast.show {
593     opacity: 1;
594     transform: translateY(0);
595 }
596
597 @media (max-width: 1180px) {
598     main { grid-template-columns: 1fr; }
599     .control-panel { max-height: none; }
600     .summary-strip { grid-template-columns: repeat(2, 1fr); }
601     .algo-flow { grid-template-columns: repeat(3, 1fr); }
602     .metrics-viz-grid { grid-template-columns: 1fr; }
603     .principle-grid { grid-template-columns: 1fr; }
604 }
605
606 @media (max-width: 760px) {
607     header { grid-template-columns: 1fr; }
608     .toolbar { justify-content: flex-start; }
609     .chart-grid, .report-grid, .summary-strip, .metrics-viz-grid, .principle-grid { grid-template-
610 columns: 1fr; }
611     .field-grid, .check-list { grid-template-columns: 1fr; }
612     .algo-flow { grid-template-columns: 1fr; }
613     .flow-step::after { display: none; }
614     h1 { white-space: normal; }
615 }
616 </style>
617 </head>
618 <body>
619     <div class="app-shell">
620         <header>
621             <div class="brand">
622                 <div class="brand-mark">H</div>
623                 <div>
624                     <h1>双线性 Hall 高精度位置检测与补偿软件</h1>
625                     <div class="subtitle">参数配置 · 信号生成 · 误差注入 · 补偿算法 · 工况仿
626 真 · 结果导出</div>
627                 </div>
628             </div>
629             <div class="toolbar">
630                 <button class="primary" id="runBtn">运行仿真</button>
631                 <button id="resetBtn">恢复默认</button>
632                 <button id="exportPngBtn">导出 PNG</button>
633                 <button id="exportCsvBtn">导出 CSV</button>
634                 <button id="exportParamBtn">导出参数</button>
635             </div>
636         </header>

```

```

637
638     <main>
639     <aside>
640         <div class="panel control-panel">
641             <section class="section">
642                 <h2>参数配置 <span class="tag">Hall / ADC</span></h2>
643                 <div class="field-grid">
644                     <label class="field">Hall 极对数 p
645                     <input id="polePairs" type="number" min="1" max="32" step="1" value="6">
646                     </label>
647                     <label class="field">机械转速 / rpm
648                     <div class="range-line wide">
649                         <input id="speedRpmSlider" type="range" min="1" max="3000" step="1"
650 value="100">
651                         <input id="speedRpm" type="number" min="0.1" max="3000" step="1"
652 value="100">
653                     </div>
654                     </label>
655                     <label class="field">仿真时长 / s
656                     <input id="simTime" type="number" min="0.1" max="5" step="0.1"
657 value="0.5">
658                     </label>
659                     <label class="field">采样频率 / Hz
660                     <input id="sampleRate" type="number" min="500" max="50000" step="500"
661 value="5000">
662                     </label>
663                     <label class="field">ADC 位数
664                     <input id="adcBits" type="number" min="8" max="24" step="1" value="16">
665                     </label>
666                     <label class="field">参考电压 / V
667                     <input id="vref" type="number" min="1" max="5" step="0.1" value="3.3">
668                     </label>
669                     <label class="field">Hall 安装夹角 / deg
670                     <div class="range-line wide">
671                         <input id="mountAngleDegSlider" type="range" min="60" max="120"
672 step="0.5" value="90">
673                         <input id="mountAngleDeg" type="number" min="60" max="120"
674 step="0.5" value="90">
675                     </div>
676                     </label>
677                 </div>
678             </section>
679
680             <section class="section">
681                 <h2>误差注入 <span class="tag">可开关</span></h2>
682                 <div class="check-list">
683                     <label class="check-item"><input id="enableOffset" type="checkbox" checked>
684 零偏</label>
685                     <label class="check-item"><input id="enableAmp" type="checkbox" checked>
686 幅值不一致</label>
687                     <label class="check-item"><input id="enablePhase" type="checkbox" checked>
688 相位不正交</label>
689                     <label class="check-item"><input id="enableHarm" type="checkbox" checked>

```

```

690 谐波</label>
691      <label class="check-item"><input id="enableNoise" type="checkbox" checked>
692 白噪声</label>
693      <label class="check-item"><input id="enableAdc" type="checkbox" checked>
694 ADC 量化</label>
695     </div>
696     <div class="field-grid">
697         <label class="field">正弦零偏误差 / V
698             <input id="offsetS" type="number" step="0.01" value="0.05">
699         </label>
700         <label class="field">余弦零偏误差 / V
701             <input id="offsetC" type="number" step="0.01" value="-0.08">
702         </label>
703         <label class="field">正弦幅值 As / V
704             <input id="ampS" type="number" min="0.1" step="0.01" value="1.00">
705         </label>
706         <label class="field">余弦幅值 Ac / V
707             <input id="ampC" type="number" min="0.1" step="0.01" value="0.85">
708         </label>
709         <label class="field">相位误差 / deg
710             <div class="range-line wide">
711                 <input id="phaseDegSlider" type="range" min="-30" max="30" step="0.5"
712 value="5">
713                 <input id="phaseDeg" type="number" step="0.5" value="5">
714             </div>
715         </label>
716         <label class="field">噪声 RMS / V
717             <div class="range-line wide">
718                 <input id="noiseRmsSlider" type="range" min="0" max="0.08"
719 step="0.001" value="0.010">
720                 <input id="noiseRms" type="number" min="0" step="0.001" value="0.010">
721             </div>
722         </label>
723     </div>
724     <label class="field">谐波强度
725         <div class="range-line">
726             <input id="harmScale" type="range" min="0" max="2" step="0.05" value="1">
727             <input id="harmScaleValue" type="number" min="0" max="2" step="0.05"
728 value="1">
729         </div>
730     </label>
731 </section>
732
733 <section class="section">
734     <h2>补偿算法 <span class="tag">实时解算</span></h2>
735     <div class="field-grid">
736         <label class="field">LPF 截止频率 / Hz
737             <div class="range-line wide">
738                 <input id="lpfHzSlider" type="range" min="1" max="2000" step="1"
739 value="300">
740                 <input id="lpfHz" type="number" min="1" max="2000" step="10"
741 value="300">
742             </div>

```

```
743         </label>
744         <label class="field">alpha-beta  $\alpha$ 
745         <div class="range-line wide">
746             <input id="abAlphaSlider" type="range" min="0" max="1" step="0.01"
747 value="0.22">
748             <input id="abAlpha" type="number" min="0" max="1" step="0.01"
749 value="0.22">
750         </div>
751         </label>
752         <label class="field">alpha-beta  $\beta$ 
753         <div class="range-line wide">
754             <input id="abBetaSlider" type="range" min="0" max="0.25" step="0.001"
755 value="0.018">
756             <input id="abBeta" type="number" min="0" max="1" step="0.001"
757 value="0.018">
758         </div>
759         </label>
760         <label class="field">ESO 带宽 / Hz
761         <div class="range-line wide">
762             <input id="esoHzSlider" type="range" min="5" max="500" step="5"
763 value="60">
764             <input id="esoHz" type="number" min="5" max="500" step="5"
765 value="60">
766         </div>
767         </label>
768         <label class="field">信号补偿阶次
769             <input id="signalOrders" type="text" value="2,3,5,6">
770         </label>
771         <label class="field">角度补偿阶次
772             <input id="angleOrders" type="text" value="1,2,3,4,5,6,7,8">
773         </label>
774     </div>
775 </section>
776
777 <section class="section">
778     <h2>工况仿真 <span class="tag">云台场景</span></h2>
779     <label class="field">工况类型
780         <select id="scenario">
781             <option value="constant">恒速</option>
782             <option value="variable">变速</option>
783             <option value="impact">冲击</option>
784             <option value="load">负载扰动</option>
785         </select>
786     </label>
787     <div class="field-grid">
788         <label class="field">变速目标倍率
789             <div class="range-line wide">
790                 <input id="speedRatioSlider" type="range" min="0.2" max="5" step="0.1"
791 value="1.8">
792                 <input id="speedRatio" type="number" min="0.2" max="5" step="0.1"
793 value="1.8">
794             </div>
795         </label>
```

```
796         <label class="field">冲击幅值 / deg
797         <div class="range-line wide">
798             <input id="impactDegSlider" type="range" min="0" max="20" step="0.5"
799 value="2">
800             <input id="impactDeg" type="number" min="0" max="20" step="0.5"
801 value="2">
802         </div>
803     </label>
804     <label class="field">负载降速比例
805     <div class="range-line wide">
806         <input id="loadDropSlider" type="range" min="0" max="0.8" step="0.05"
807 value="0.25">
808         <input id="loadDrop" type="number" min="0" max="0.8" step="0.05"
809 value="0.25">
810     </div>
811     </label>
812     <label class="field">随机种子
813     <input id="seed" type="number" step="1" value="1808">
814     </label>
815 </div>
816 </section>
817 </div>
818 </aside>
819
820 <section class="workspace">
821     <div class="summary-strip">
822         <div class="metric">
823             <div class="name">原始解角最大误差</div>
824             <div class="value" id="rawMax">--</div>
825             <div class="note">atan2 直接解算</div>
826         </div>
827         <div class="metric">
828             <div class="name">基础补偿 RMS</div>
829             <div class="value" id="basicRms">--</div>
830             <div class="note">零偏 / 幅值 / 相位</div>
831         </div>
832         <div class="metric">
833             <div class="name">谐波补偿 RMS</div>
834             <div class="value" id="harmRms">--</div>
835             <div class="note">信号域 + 角度域</div>
836         </div>
837         <div class="metric">
838             <div class="name">ESO RMS</div>
839             <div class="value" id="esoRms">--</div>
840             <div class="note">最终角度估计</div>
841         </div>
842     </div>
843
844     <div class="tabs" role="tablist">
845         <button class="tab active" data-view="dashboard">结果显示</button>
846         <button class="tab" data-view="metrics">误差指标</button>
847         <button class="tab" data-view="principle">原理介绍</button>
848         <button class="tab" data-view="flow">算法流程</button>
```



```

849     <button class="tab" data-view="report">报告导出</button>
850 </div>
851
852 <div id="dashboard" class="view active">
853     <div class="chart-grid">
854         <div class="chart-card wide">
855             <div class="chart-head">
856                 <div class="chart-title">电机与双线性 Hall 安装三维示意</div>
857                 <div class="legend">
858                     <span style="color:#57a9df"><i class="swatch"></i> 中空磁钢 / 转子
859 </span>
860                     <span style="color:#d9812b"><i class="swatch"></i> Hall S</span>
861                     <span style="color:#2ba86a"><i class="swatch"></i> Hall C , 空间约
862 90°</span>
863                 </div>
864             </div>
865             <canvas id="modelCanvas" class="model-canvas"></canvas>
866         </div>
867         <div class="chart-card">
868             <div class="chart-head">
869                 <div class="chart-title">Hall 电压波形</div>
870                 <div class="legend">
871                     <span style="color:#1d65d8"><i class="swatch"></i> Hs</span>
872                     <span style="color:#d23b3b"><i class="swatch"></i> Hc</span>
873                 </div>
874             </div>
875             <canvas id="voltageCanvas"></canvas>
876         </div>
877         <div class="chart-card">
878             <div class="chart-head">
879                 <div class="chart-title">Lissajous 相图</div>
880                 <div class="legend">
881                     <span style="color:#b6c1c9"><i class="swatch"></i> 原始</span>
882                     <span style="color:#1b1f23"><i class="swatch"></i> 补偿后</span>
883                 </div>
884             </div>
885             <canvas id="lissajousCanvas"></canvas>
886         </div>
887         <div class="chart-card">
888             <div class="chart-head">
889                 <div class="chart-title">FFT 主要谐波</div>
890                 <div class="legend">
891                     <span style="color:#7a55d9"><i class="swatch"></i> 幅值</span>
892                 </div>
893             </div>
894             <canvas id="fftCanvas"></canvas>
895         </div>
896         <div class="chart-card">
897             <div class="chart-head">
898                 <div class="chart-title">机械角解算误差</div>
899                 <div class="legend">
900                     <span style="color:#d23b3b"><i class="swatch"></i> 原始</span>
901                     <span style="color:#25855b"><i class="swatch"></i> 谐波</span>

```

```

902         <span style="color:#7c3fb0"><i class="swatch"></i>ESO</span>
903     </div>
904 </div>
905     <canvas id="errorCanvas"></canvas>
906 </div>
907 </div>
908 </div>
909
910 <div id="metrics" class="view">
911     <div class="metrics-viz-grid">
912         <div class="chart-card">
913             <div class="chart-head">
914                 <div class="chart-title">误差指标柱状图</div>
915                 <div class="legend">
916                     <span style="color:#57a9df"><i class="swatch"></i>最大误差</span>
917                     <span style="color:#d9812b"><i class="swatch"></i>RMS 误差</span>
918                 </div>
919             </div>
920             <canvas id="metricsBarCanvas"></canvas>
921         </div>
922         <div class="chart-card">
923             <div class="chart-head">
924                 <div class="chart-title">补偿效果雷达图</div>
925                 <div class="legend">
926                     <span style="color:#1672ad"><i class="swatch"></i> 误差越小越靠外
927 </span>
928                 </div>
929             </div>
930             <canvas id="metricsRadarCanvas"></canvas>
931         </div>
932     </div>
933     <div class="table-wrap">
934         <table>
935             <thead>
936                 <tr>
937                     <th>处理方式</th>
938                     <th>最大误差 / deg</th>
939                     <th>RMS 误差 / deg</th>
940                     <th>说明</th>
941                 </tr>
942             </thead>
943             <tbody id="metricsBody"></tbody>
944         </table>
945     </div>
946 </div>
947
948 <div id="principle" class="view">
949     <div class="principle-grid">
950         <div class="principle-card">
951             <h3>1. 双线性 Hall 角度检测模型</h3>
952             <p>两只线性 Hall 固定在电机端面附近,理想情况下空间夹角为 90°。转子
953 磁场扫过传感器时,两路输出形成近似正余弦信号,经过归一化后可通过 atan2 解算磁场
954 角,再换算为机械角。</p>

```

```

955         <div class="formula"> $\theta_{\text{mag}} = p \cdot \theta_m +$ 
956  $\theta_0$ </div>
957         <div class="formula"> $H_s = O_s + A_s \sin(\theta_{\text{mag}}) + n_s$ </div>
958 sin( $\theta_{\text{mag}}$ ) +  $n_s$ </div>
959         <div class="formula"> $H_c = O_c + A_c \cos(\theta_{\text{mag}} + \Delta\phi) + n_c$ </div>
960 cos( $\theta_{\text{mag}}$  +  $\Delta\phi$ ) +  $n_c$ </div>
961         <p>其中  $p$  是 Hall 磁极对数,  $O$  是零偏,  $A$  是幅值,  $\Delta\phi$  包含安装夹角偏
962 离  $90^\circ$  和额外相位误差。</p>
963     </div>
964     <div class="principle-card">
965         <h3>2. 误差来源</h3>
966         <p>真实系统中, 两路 Hall 不会天然构成标准单位圆, 常见误差会把
967 Lissajous 相图从圆变成偏心圆、椭圆、倾斜椭圆或带波纹的闭合曲线。</p>
968         <ul class="mini-list">
969             <li>零偏误差: 圆心偏离原点。</li>
970             <li>幅值不一致: 圆被拉伸成椭圆。</li>
971             <li>安装角不为  $90^\circ$ : 相图倾斜, 解角产生周期误差。</li>
972             <li>磁钢偏心/充磁不均: 出现 2、3、5、6 次等高次谐波。</li>
973             <li>ADC 量化与白噪声: 带来随机解角抖动。</li>
974         </ul>
975     </div>
976     <div class="principle-card">
977         <h3>3. 确定性补偿链路</h3>
978         <p>确定性补偿先处理可标定、可重复的误差: 零偏、幅值、相位和谐波。
979 其目标是把两路信号重新拉回标准正余弦关系。</p>
980         <div class="formula"> $x_s = (H_s - O_s) /$ 
981  $A_s$ , &nbsp;  $x_c = (H_c - O_c) /$ 
982  $A_c$ </div>
983         <div class="formula"> $x_{c,\text{corr}} = (x_c + x_s \sin\Delta\phi) / \cos\Delta\phi$ </div>
984 sin $\Delta\phi$  / cos $\Delta\phi$ </div>
985         <div
986 class="formula"> $e_{\text{fit}}(\theta) = \Sigma [A_k \sin(k\theta) + B_k \cos(k\theta)]$ </div>
987 <p>补偿后再用 atan2 输出角度:  $\theta = \text{atan2}(x_s, x_{c,\text{corr}})$ 。</p>
988 </div>
989     <div class="principle-card">
990         <h3>4. 动态估计与滤波</h3>
991         <p>LPF、 $\alpha$ - $\beta$  和 ESO 面向随机噪声、采样误差和动态扰动。LPF 简单但有
992 滞后;  $\alpha$ - $\beta$  用“角度-速度”预测提高动态跟随; ESO 进一步估计未建模扰动。</p>
993         <div class="formula">LPF: &nbsp;  $y[k] = \alpha y[k-1] + (1-\alpha)x[k]$ </div>
994         <div class="formula"> $\alpha$ - $\beta$ : &nbsp;  $\theta^{\sup} = \theta[k-1] + T_s \omega[k-1]$ , &nbsp;
995  $e = \theta_{\text{meas}} - \theta^{\sup}$ </div>
996         <div class="formula">ESO: &nbsp;  $z_1 \approx \theta$ , &nbsp;
997  $z_2 \approx \omega$ , &nbsp;  $z_3 \approx d$ </div>
998         <p>所以本软件把确定性补偿和动态估计分开展示, 方便判断误差到底来自
999 结构、磁场还是采样噪声。</p>
1000     </div>
1001 </div>
1002 </div>
1003 </div>
1004
1005 <div id="flow" class="view">
1006     <div class="algo-flow">
1007         <div class="flow-step"><div class="step-no">1</div><div class="step-name">两

```

```

1008 路 Hall</div><div class="step-note">采集 Hs/Hc。模型：H=O+A·sin/cos(θ)+谐波+噪声。
1009 </div></div>
1010     <div class="flow-step"><div class="step-no">2</div><div class="step-name">零
1011 偏与幅值归一化</div><div class="step-note">x=(H-O)/A，把偏心圆和幅值椭圆先拉回单位尺
1012 度。</div></div>
1013     <div class="flow-step"><div class="step-no">3</div><div class="step-name">相
1014 位 / 安 装 角 补 偿 </div><div class="step-note">用
1015  $x_{corr} = (x_c + x_s \sin \Delta \varphi) / \cos \Delta \varphi$  修正非 90° 安装。
1016 </div></div>
1017     <div class="flow-step"><div class="step-no">4</div><div class="step-name">谐
1018 波重构补偿</div><div class="step-note">用  $\Sigma(A_k \sin k\theta + B_k \cos k\theta)$  拟合周期误差并扣除。
1019 </div></div>
1020     <div class="flow-step"><div class="step-no">5</div><div class="step-
1021 name">atan2 解角</div><div class="step-note"> $\theta_{mag} = \text{atan2}(x_s, x_c)$ ，机械角
1022  $\theta_m = \theta_{mag}$ 。</div></div>
1023     <div class="flow-step"><div class="step-no">6</div><div class="step-name"> $\alpha$ - $\beta$ 
1024 / ESO 动态估计</div><div class="step-note"> $\alpha$ - $\beta$  估计角度速度，ESO 进一步估计扰动  $z_3 \approx d$ 。
1025 </div></div>
1026 </div>
1027 </div>
1028
1029 <div id="report" class="view">
1030     <div class="report-grid">
1031         <div class="report-box">
1032             <h3>软件功能模块</h3>
1033             <p>本软件包括参数配置、信号生成、误差注入、补偿算法、工况仿真、结
1034 果显示和报告导出七个模块。</p>
1035             <p>当前模型支持零偏、幅值不一致、相位不正交、2/3/5/6 次谐波、ADC 量
1036 化、白噪声、恒速、变速、冲击和负载扰动等工况。</p>
1037             <p>补偿链路采用零偏估计、幅值归一化、相位补偿、谐波重构、角度域傅
1038 里叶误差补偿、 $\alpha$ - $\beta$  状态估计和离散 ESO 估计。</p>
1039         </div>
1040         <div class="report-box">
1041             <h3>导出内容</h3>
1042             <p>PNG：导出四张核心结果图组成的汇总图。</p>
1043             <p>CSV：导出各处理方式的误差和 RMS 误差。</p>
1044             <p>参数：导出当前仿真参数 JSON，可用于复现实验配置。</p>
1045             <div style="display:flex;gap:8px;flex-wrap:wrap;margin-top:12px">
1046                 <button id="reportPngBtn">导出汇总 PNG</button>
1047                 <button id="reportCsvBtn">导出误差 CSV</button>
1048                 <button id="reportParamBtn">导出参数 JSON</button>
1049             </div>
1050         </div>
1051     </div>
1052 </div>
1053 </section>
1054 </main>
1055 </div>
1056 <div class="toast" id="toast"></div>
1057
1058 <script>
1059     "use strict";
1060

```

```
1061     const ids = [  
1062         "polePairs", "speedRpm", "simTime", "sampleRate", "adcBits", "vref", "mountAngleDeg",  
1063         "enableOffset", "enableAmp", "enablePhase", "enableHarm", "enableNoise", "enableAdc",  
1064         "offsetS", "offsetC", "ampS", "ampC", "phaseDeg", "noiseRms", "harmScale",  
1065         "harmScaleValue",  
1066         "lpfHz", "abAlpha", "abBeta", "esoHz", "signalOrders", "angleOrders",  
1067         "scenario", "speedRatio", "impactDeg", "loadDrop", "seed"  
1068     ];  
1069     const $ = (id) => document.getElementById(id);  
1070     const state = { result: null, params: null, modelTheta: 0, modelLastTs: 0, modelAnimReq: 0 };  
1071  
1072     const colors = {  
1073         blue: "#1d65d8",  
1074         red: "#d23b3b",  
1075         green: "#25855b",  
1076         purple: "#7c3fb0",  
1077         grey: "#b6c1c9",  
1078         orange: "#d9812b",  
1079         black: "#1b1f23",  
1080         axis: "#8da8b9",  
1081         grid: "#e2eef6"  
1082     };  
1083  
1084     function number(id) {  
1085         const v = Number($(id).value);  
1086         return Number.isFinite(v) ? v : 0;  
1087     }  
1088  
1089     function checked(id) {  
1090         return $(id).checked;  
1091     }  
1092  
1093     function parseOrders(text) {  
1094         return text.split(",")  
1095             .map(v => Number(v.trim()))  
1096             .filter(v => Number.isFinite(v) && v > 0);  
1097     }  
1098  
1099     function readParams() {  
1100         const p = {  
1101             polePairs: Math.max(1, Math.round(number("polePairs"))),  
1102             speedRpm: Math.max(0.1, number("speedRpm")),  
1103             simTime: clamp(number("simTime"), 0.1, 5),  
1104             sampleRate: clamp(number("sampleRate"), 500, 50000),  
1105             adcBits: clamp(Math.round(number("adcBits")), 8, 24),  
1106             vref: clamp(number("vref"), 1, 5),  
1107             mountAngleDeg: clamp(number("mountAngleDeg"), 60, 120),  
1108             enableOffset: checked("enableOffset"),  
1109             enableAmp: checked("enableAmp"),  
1110             enablePhase: checked("enablePhase"),  
1111             enableHarm: checked("enableHarm"),  
1112             enableNoise: checked("enableNoise"),  
1113             enableAdc: checked("enableAdc"),
```

```

1114     offsetS: number("offsetS"),
1115     offsetC: number("offsetC"),
1116     ampS: Math.max(0.1, number("ampS")),
1117     ampC: Math.max(0.1, number("ampC")),
1118     phaseDeg: number("phaseDeg"),
1119     noiseRms: Math.max(0, number("noiseRms")),
1120     harmScale: Math.max(0, number("harmScale")),
1121     lpfHz: Math.max(1, number("lpfHz")),
1122     abAlpha: clamp(number("abAlpha"), 0, 1),
1123     abBeta: clamp(number("abBeta"), 0, 1),
1124     esoHz: Math.max(5, number("esoHz")),
1125     signalOrders: parseOrders$("signalOrders").value,
1126     angleOrders: parseOrders$("angleOrders").value,
1127     scenario: $("scenario").value,
1128     speedRatio: Math.max(0.2, number("speedRatio")),
1129     impactDeg: Math.max(0, number("impactDeg")),
1130     loadDrop: clamp(number("loadDrop"), 0, 0.8),
1131     seed: Math.round(number("seed"))
1132 };
1133 p.offsetNom = p.vref / 2;
1134 p.ts = 1 / p.sampleRate;
1135 p.n = Math.min(Math.floor(p.simTime * p.sampleRate) + 1, 25000);
1136 p.actualAmpS = p.enableAmp ? p.ampS : 1.0;
1137 p.actualAmpC = p.enableAmp ? p.ampC : 1.0;
1138 p.actualOffsetS = p.offsetNom + (p.enableOffset ? p.offsetS : 0);
1139 p.actualOffsetC = p.offsetNom + (p.enableOffset ? p.offsetC : 0);
1140 p.mountErrDeg = p.mountAngleDeg - 90;
1141 p.totalPhaseDeg = p.mountErrDeg + (p.enablePhase ? p.phaseDeg : 0);
1142 p.phaseRad = deg2rad(p.totalPhaseDeg);
1143 p.harmScale = p.enableHarm ? p.harmScale : 0;
1144 return p;
1145 }
1146
1147 function simulate() {
1148     const p = readParams();
1149     const rng = mulberry32(p.seed);
1150     const t = new Array(p.n);
1151     const thetaM = new Array(p.n);
1152     const omegaM = new Array(p.n);
1153     const thetaMag = new Array(p.n);
1154     const hsClean = new Array(p.n);
1155     const hcClean = new Array(p.n);
1156     const hs = new Array(p.n);
1157     const hc = new Array(p.n);
1158     const baseOmega = 2 * Math.PI * (p.speedRpm / 60);
1159     let theta = 0;
1160
1161     for (let i = 0; i < p.n; i++) {
1162         const ti = i * p.ts;
1163         t[i] = ti;
1164         const profile = speedProfile(p, ti);
1165         omegaM[i] = profile.omega;
1166         if (i > 0) theta += 0.5 * (omegaM[i] + omegaM[i - 1]) * p.ts;

```

```

1167     thetaM[i] = theta + profile.angleOffset;
1168     thetaMag[i] = p.polePairs * thetaM[i];
1169
1170     const clean = hallSignal(thetaMag[i], p, false);
1171     hsClean[i] = clean.hs;
1172     hcClean[i] = clean.hc;
1173
1174     const noisy = hallSignal(thetaMag[i], p, true, rng);
1175     hs[i] = p.enableAdc ? adcQuantize(noisy.hs, p) : noisy.hs;
1176     hc[i] = p.enableAdc ? adcQuantize(noisy.hc, p) : noisy.hc;
1177 }
1178
1179 const cal = estimateCalibration(hs, hc, thetaMag, p);
1180 const raw = rawDecode(hs, hc, thetaMag, p);
1181 const basic = basicCompensate(hs, hc, thetaMag, cal, p);
1182 const harmonic = harmonicCompensate(basic, thetaMag, p);
1183 const thetaAb = alphaBeta(harmonic.thetaAngleComp, p.ts, p.abAlpha, p.abBeta).theta;
1184 const thetaEso = esoFilter(harmonic.thetaAngleComp, p.ts, p.esoHz).theta;
1185 const thetaLpf = firstOrderLpf(harmonic.thetaAngleComp, lpfAlpha(p.ts, p.lpfHz));
1186
1187 const errRaw = angleError(raw.theta, thetaMag, p.polePairs);
1188 const errBasic = angleError(basic.thetaBasic, thetaMag, p.polePairs);
1189 const errHarm = angleError(harmonic.thetaAngleComp, thetaMag, p.polePairs);
1190 const errLpf = angleError(thetaLpf, thetaMag, p.polePairs);
1191 const errAb = angleError(thetaAb, thetaMag, p.polePairs);
1192 const errEso = angleError(thetaEso, thetaMag, p.polePairs);
1193
1194 const fft = harmonicSpectrum(basic.sinNorm, thetaMag, 12, p);
1195 const metrics = [
1196     rowMetric("原始 atan2", errRaw, "未补偿, 直接按名义零偏和幅值解角"),
1197     rowMetric("零偏/幅值/相位补偿", errBasic, "估计中心、幅值和相位误差"),
1198     rowMetric("谐波补偿", errHarm, "信号域谐波重构 + 角度域补偿"),
1199     rowMetric("角度 LPF", errLpf, "补偿后角度一阶低通"),
1200     rowMetric("alpha-beta", errAb, "角度-角速度状态估计"),
1201     rowMetric("离散 ESO", errEso, "角度、角速度、总扰动估计")
1202 ];
1203
1204 state.params = p;
1205 state.result = { p, t, thetaM, omegaM, thetaMag, hsClean, hcClean, hs, hc, raw, basic,
1206 harmonic, thetaLpf, thetaAb, thetaEso, errRaw, errBasic, errHarm, errLpf, errAb, errEso, fft,
1207 metrics };
1208 renderAll();
1209 }
1210
1211 function speedProfile(p, t) {
1212     const base = 2 * Math.PI * (p.speedRpm / 60);
1213     let omega = base;
1214     let angleOffset = 0;
1215     if (p.scenario === "variable") {
1216         const s = smoothStep((t - 0.2 * p.simTime) / (0.6 * p.simTime));
1217         omega = base * (1 + (p.speedRatio - 1) * s) + 0.06 * base * Math.sin(2 * Math.PI * 3 * t)
1218 * s;
1219     } else if (p.scenario === "impact") {

```

```

1220     const start = 0.45 * p.simTime;
1221     if (t >= start) {
1222         const x = t - start;
1223         angleOffset = deg2rad(p.impactDeg) * Math.exp(-18 * x) * Math.sin(2 * Math.PI * 24
1224 * x);
1225     }
1226 } else if (p.scenario === "load") {
1227     const start = 0.45 * p.simTime;
1228     const s = t >= start ? 1 - Math.exp(-9 * (t - start)) : 0;
1229     omega = base * (1 - p.loadDrop * s) + 0.04 * base * Math.sin(2 * Math.PI * 7 * t) * s;
1230 }
1231 return { omega, angleOffset };
1232 }
1233
1234 function hallSignal(theta, p, withNoise, rng) {
1235     const harm = harmonicVoltage(theta, p);
1236     const hs = p.actualOffsetS + p.actualAmpS * Math.sin(theta) + harm.hs + (withNoise &&
1237 p.enableNoise ? p.noiseRms * randn(rng) : 0);
1238     const hc = p.actualOffsetC + p.actualAmpC * Math.cos(theta + p.phaseRad) + harm.hc +
1239 (withNoise && p.enableNoise ? p.noiseRms * randn(rng) : 0);
1240     return { hs, hc };
1241 }
1242
1243 function harmonicVoltage(theta, p) {
1244     const sSin = [0.050, -0.035, 0.024, 0.016];
1245     const sCos = [0.018, 0.028, -0.014, 0.010];
1246     const cSin = [-0.032, 0.022, 0.016, -0.012];
1247     const cCos = [0.040, 0.030, -0.018, 0.013];
1248     const orders = [2, 3, 5, 6];
1249     let hs = 0;
1250     let hc = 0;
1251     for (let i = 0; i < orders.length; i++) {
1252         const n = orders[i];
1253         hs += sSin[i] * Math.sin(n * theta) + sCos[i] * Math.cos(n * theta);
1254         hc += cSin[i] * Math.sin(n * theta) + cCos[i] * Math.cos(n * theta);
1255     }
1256     return { hs: p.harmScale * hs, hc: p.harmScale * hc };
1257 }
1258
1259 function rawDecode(hs, hc, thetaMag, p) {
1260     const s = hs.map(v => (v - p.offsetNom) / 1.0);
1261     const c = hc.map(v => (v - p.offsetNom) / 1.0);
1262     return { theta: unwrap(s.map((v, i) => Math.atan2(v, c[i]))), sinNorm: s, cosNorm: c };
1263 }
1264
1265 function estimateCalibration(hs, hc, thetaMag, p) {
1266     const os = 0.5 * (max(hs) + min(hs));
1267     const oc = 0.5 * (max(hc) + min(hc));
1268     const as = Math.max(1e-9, 0.5 * (max(hs) - min(hs)));
1269     const ac = Math.max(1e-9, 0.5 * (max(hc) - min(hc)));
1270     const s = hs.map(v => (v - os) / as);
1271     const c = hc.map(v => (v - oc) / ac);
1272     const sm = mean(s);

```



```

1273     const cm = mean(c);
1274     const cov = mean(s.map((v, i) => (v - sm) * (c[i] - cm)));
1275     const den = Math.sqrt(mean(s.map(v => (v - sm) ** 2)) * mean(c.map(v => (v - cm) ** 2)));
1276     const coef = clamp(cov / Math.max(den, 1e-12), -1, 1);
1277     const phase = Math.asin(-coef);
1278     return { os, oc, as, ac, phase };
1279 }
1280
1281 function basicCompensate(hs, hc, thetaMag, cal, p) {
1282     const sinNorm = hs.map(v => (v - cal.os) / cal.as);
1283     const cosRaw = hc.map(v => (v - cal.oc) / cal.ac);
1284     const cosNorm = cosRaw.map((v, i) => (v + sinNorm[i] * Math.sin(cal.phase)) /
1285 Math.cos(cal.phase));
1286     const thetaBasic = alignAngle(unwrap(sinNorm.map((v, i) => Math.atan2(v, cosNorm[i]))),
1287 thetaMag);
1288     return { thetaBasic, sinNorm, cosNorm };
1289 }
1290
1291 function harmonicCompensate(basic, thetaMag, p) {
1292     const phiSig = basisMatrix(basic.thetaBasic, p.signalOrders);
1293     const sinIdeal = thetaMag.map(Math.sin);
1294     const cosIdeal = thetaMag.map(Math.cos);
1295     const coefS = leastSquares(phiSig, basic.sinNorm.map((v, i) => v - sinIdeal[i]));
1296     const coefC = leastSquares(phiSig, basic.cosNorm.map((v, i) => v - cosIdeal[i]));
1297     const sinEst = multiply(phiSig, coefS);
1298     const cosEst = multiply(phiSig, coefC);
1299     const sinSig = basic.sinNorm.map((v, i) => v - sinEst[i]);
1300     const cosSig = basic.cosNorm.map((v, i) => v - cosEst[i]);
1301     const thetaSig = alignAngle(unwrap(sinSig.map((v, i) => Math.atan2(v, cosSig[i]))),
1302 thetaMag);
1303
1304     const phiAng = basisMatrix(basic.thetaBasic, p.angleOrders);
1305     const errBasic = basic.thetaBasic.map((v, i) => wrapPi(v - thetaMag[i]));
1306     const coefAng = leastSquares(phiAng, errBasic);
1307     const errHat = multiply(phiAng, coefAng);
1308     const thetaAngleComp = alignAngle(basic.thetaBasic.map((v, i) => v - errHat[i]), thetaMag);
1309     return { thetaSig, thetaAngleComp, sinSig, cosSig, coefS, coefC, coefAng };
1310 }
1311
1312 function alphaBeta(thetaMeas, ts, alpha, beta) {
1313     const n = thetaMeas.length;
1314     const theta = new Array(n).fill(0);
1315     const omega = new Array(n).fill(0);
1316     theta[0] = thetaMeas[0];
1317     omega[0] = n > 1 ? (thetaMeas[1] - thetaMeas[0]) / ts : 0;
1318     for (let i = 1; i < n; i++) {
1319         const pred = theta[i - 1] + ts * omega[i - 1];
1320         const err = thetaMeas[i] - pred;
1321         theta[i] = pred + alpha * err;
1322         omega[i] = omega[i - 1] + beta / ts * err;
1323     }
1324     return { theta, omega };
1325 }

```

```

1326
1327 function esoFilter(thetaMeas, ts, bandwidthHz) {
1328     const n = thetaMeas.length;
1329     const z1 = new Array(n).fill(0);
1330     const z2 = new Array(n).fill(0);
1331     const z3 = new Array(n).fill(0);
1332     const wo = 2 * Math.PI * bandwidthHz;
1333     const b1 = 3 * wo;
1334     const b2 = 3 * wo * wo;
1335     const b3 = wo * wo * wo;
1336     z1[0] = thetaMeas[0];
1337     for (let i = 1; i < n; i++) {
1338         const err = thetaMeas[i - 1] - z1[i - 1];
1339         z1[i] = z1[i - 1] + ts * (z2[i - 1] + b1 * err);
1340         z2[i] = z2[i - 1] + ts * (z3[i - 1] + b2 * err);
1341         z3[i] = z3[i - 1] + ts * b3 * err;
1342     }
1343     return { theta: z1, omega: z2, disturbance: z3 };
1344 }
1345
1346 function harmonicSpectrum(signal, thetaMag, maxOrder, p) {
1347     const x = signal.map(v => v - mean(signal));
1348     const result = [];
1349     for (let n = 1; n <= maxOrder; n++) {
1350         let ss = 0, cc = 0;
1351         for (let i = 0; i < x.length; i++) {
1352             ss += x[i] * Math.sin(n * thetaMag[i]);
1353             cc += x[i] * Math.cos(n * thetaMag[i]);
1354         }
1355         const amp = 2 / x.length * Math.sqrt(ss * ss + cc * cc);
1356         result.push({ order: n, freq: n * p.polePairs * p.speedRpm / 60, amp, db: 20 *
1357 Math.log10(Math.max(amp, 1e-6)) });
1358     }
1359     return result;
1360 }
1361
1362 function renderAll() {
1363     const r = state.result;
1364     if (!r) return;
1365     $("rawMax").textContent = fmtDeg(maxAbs(r.errRaw));
1366     $("basicRms").textContent = fmtDeg(rms(r.errBasic));
1367     $("harmRms").textContent = fmtDeg(rms(r.errHarm));
1368     $("esoRms").textContent = fmtDeg(rms(r.errEso));
1369     renderMetrics();
1370     drawMotorModel();
1371     drawVoltage();
1372     drawLissajous();
1373     drawFft();
1374     drawError();
1375     drawMetricsVisuals();
1376 }
1377
1378 function renderMetrics() {

```

```

1379     const body = $("metricsBody");
1380     body.innerHTML = "";
1381     for (const m of state.result.metrics) {
1382         const tr = document.createElement("tr");
1383         tr.innerHTML
1384         `<td>${m.name}</td><td>${m.maxDeg.toFixed(4)}</td><td>${m.rmsDeg.toFixed(4)}</td><td>
1385         >${m.note}</td>`;
1386         body.appendChild(tr);
1387     }
1388 }
1389
1390 function drawVoltage() {
1391     const r = state.result;
1392     drawLines("voltageCanvas", r.t, [
1393         { y: r.hs, color: colors.blue, name: "Hs" },
1394         { y: r.hc, color: colors.red, name: "Hc" }
1395     ], { xLabel: "Time / s", yLabel: "Voltage / V" });
1396 }
1397
1398 function drawLissajous() {
1399     const r = state.result;
1400     drawXY("lissajousCanvas", [
1401         { x: r.basic.cosNorm, y: r.basic.sinNorm, color: colors.grey, name: "raw" },
1402         { x: r.harmonic.cosSig, y: r.harmonic.sinSig, color: colors.black, name: "comp" }
1403     ], { xLabel: "normalized Hc", yLabel: "normalized Hs" });
1404 }
1405
1406 function drawFft() {
1407     const r = state.result;
1408     drawBars("fftCanvas", r.fft.map(v => v.freq), r.fft.map(v => v.db), {
1409         color: "#7a55d9",
1410         xLabel: "Frequency / Hz",
1411         yLabel: "Amplitude / dB",
1412         labels: r.fft.map(v => `${v.order}`)
1413     });
1414 }
1415
1416 function drawError() {
1417     const r = state.result;
1418     drawLines("errorCanvas", r.t, [
1419         { y: r.errRaw.map(rad2deg), color: colors.red, name: "raw" },
1420         { y: r.errHarm.map(rad2deg), color: colors.green, name: "harmonic" },
1421         { y: r.errEso.map(rad2deg), color: colors.purple, name: "eso" }
1422     ], { xLabel: "Time / s", yLabel: "Mechanical error / deg" });
1423 }
1424
1425 function drawMotorModel() {
1426     const r = state.result;
1427     if (!r) return;
1428     const p = r.p;
1429     const canvas = setupCanvas("modelCanvas");
1430     const ctx = canvas.getContext("2d");
1431     clearChart(ctx, canvas);

```

```

1432     const cx = canvas.width * 0.43;
1433     const cy = canvas.height * 0.52;
1434     const rx = Math.min(canvas.width * 0.24, 190);
1435     const ry = rx * 0.46;
1436     const depth = 54;
1437
1438     const grad = ctx.createLinearGradient(cx - rx, cy - ry - depth, cx + rx, cy + ry);
1439     grad.addColorStop(0, "#cfefff");
1440     grad.addColorStop(1, "#ffffff");
1441     ctx.fillStyle = grad;
1442     ctx.fillRect(0, 0, canvas.width, canvas.height);
1443     drawGridFloor(ctx, canvas, cy + ry + 46);
1444
1445     // 伪三维电机壳体
1446     ctx.fillStyle = "#d9edf8";
1447     ctx.strokeStyle = "#8bbdd8";
1448     ctx.lineWidth = 2;
1449     ctx.beginPath();
1450     ctx.ellipse(cx, cy + depth, rx, ry, 0, 0, Math.PI * 2);
1451     ctx.fill();
1452     ctx.stroke();
1453     ctx.fillStyle = "#edf8ff";
1454     ctx.beginPath();
1455     ctx.ellipse(cx, cy, rx, ry, 0, 0, Math.PI * 2);
1456     ctx.fill();
1457     ctx.stroke();
1458     ctx.strokeStyle = "rgba(83, 143, 178, 0.45)";
1459     ctx.beginPath();
1460     ctx.moveTo(cx - rx, cy);
1461     ctx.lineTo(cx - rx, cy + depth);
1462     ctx.moveTo(cx + rx, cy);
1463     ctx.lineTo(cx + rx, cy + depth);
1464     ctx.stroke();
1465
1466     const rotorAngle = state.modelTheta || 0;
1467     const fieldAngle = rotorAngle * p.polePairs - Math.PI / 2;
1468
1469     // 中空磁钢，用交替色块表达 N/S，并随时间旋转。
1470     const poleCount = p.polePairs * 2;
1471     for (let i = 0; i < poleCount; i++) {
1472         const a0 = rotorAngle - Math.PI / 2 + i * Math.PI * 2 / poleCount;
1473         const a1 = rotorAngle - Math.PI / 2 + (i + 0.82) * Math.PI * 2 / poleCount;
1474         drawRingSector(ctx, cx, cy, rx * 0.70, ry * 0.70, rx * 0.43, ry * 0.43, a0, a1, i % 2 ?
1475             "#6ec6e8" : "#f49b69");
1476     }
1477     ctx.fillStyle = "#ffffff";
1478     ctx.strokeStyle = "#93bfd6";
1479     ctx.lineWidth = 2;
1480     ctx.beginPath();
1481     ctx.ellipse(cx, cy, rx * 0.34, ry * 0.34, 0, 0, Math.PI * 2);
1482     ctx.fill();
1483     ctx.stroke();
1484

```

```

1485 // 转轴与磁场方向。
1486 ctx.strokeStyle = "#315b73";
1487 ctx.lineWidth = 3;
1488 ctx.beginPath();
1489 ctx.moveTo(cx - rx * 1.08, cy);
1490 ctx.lineTo(cx + rx * 1.12, cy);
1491 ctx.stroke();
1492 drawArrow(ctx, cx, cy, cx + rx * 0.78 * Math.cos(fieldAngle), cy + ry * 0.78 *
1493 Math.sin(fieldAngle), "#1672ad");
1494 ctx.fillStyle = "#1672ad";
1495 ctx.font = "700 13px Microsoft YaHei";
1496 ctx.fillText("磁场角  $\theta_{mag}$ ", cx + rx * 0.32, cy - ry * 0.60);
1497
1498 // 两个线性 Hall, Hall C 位置由安装夹角控制。
1499 const hallSAngle = -Math.PI / 2;
1500 const hallCAngle = hallSAngle + deg2rad(p.mountAngleDeg);
1501 const hs = ellipsePoint(cx, cy, rx * 0.93, ry * 0.93, hallSAngle);
1502 const hc = ellipsePoint(cx, cy, rx * 0.93, ry * 0.93, hallCAngle);
1503 drawHallChip(ctx, hs.x - 30, hs.y - 56, "Hall S", "#d9812b");
1504 drawHallChip(ctx, hc.x + 16, hc.y - 18, "Hall C", "#2ba86a");
1505 drawArrow(ctx, hs.x - 2, hs.y - 23, hs.x, hs.y - 2, "#d9812b");
1506 drawArrow(ctx, hc.x + 8, hc.y + 2, hc.x - 8, hc.y, "#2ba86a");
1507 ctx.strokeStyle = "#8da8b9";
1508 ctx.setLineDash([5, 4]);
1509 ctx.beginPath();
1510 ctx.moveTo(cx, cy);
1511 ctx.lineTo(hs.x, hs.y);
1512 ctx.moveTo(cx, cy);
1513 ctx.lineTo(hc.x, hc.y);
1514 ctx.stroke();
1515 ctx.setLineDash([]);
1516 drawAngleArc(ctx, cx, cy, rx * 0.31, ry * 0.31, hallSAngle, hallCAngle, "#1672ad",
1517 `${p.mountAngleDeg.toFixed(1)}°`);
1518
1519 // 右侧信息卡, 让结构图和参数联动。
1520 const infoX = Math.max(18, Math.min(canvas.width - 270, canvas.width * 0.70));
1521 drawInfoCard(ctx, infoX, 48, 245, 190, [
1522   ["Hall 极对数", `${p.polePairs}`],
1523   ["磁极数", `${p.poleCount}`],
1524   ["安装夹角", `${p.mountAngleDeg.toFixed(1)}°`],
1525   ["等效相位误差", `${p.totalPhaseDeg.toFixed(1)}°`],
1526   ["ADC", `${p.adcBits} bit / ${p.vref.toFixed(1)} V`],
1527   ["机械转速", `${p.speedRpm.toFixed(1)} rpm`],
1528   ["工况", scenarioName(p.scenario)]
1529 ]);
1530 drawInfoCard(ctx, infoX, 254, 245, 60, [
1531   ["输出链路", "Hs/Hc → 补偿 → atan2"],
1532   ["用途", "电机轴 / 关节相对角反馈"]
1533 ]);
1534
1535 ctx.fillStyle = "#19384d";
1536 ctx.font = "700 16px Microsoft YaHei";
1537 ctx.fillText("中空磁钢 + 双线性 Hall 端面安装示意", 22, 28);

```

```

1538     ctx.font = "12px Microsoft YaHei";
1539     ctx.fillStyle = "#60788a";
1540     ctx.fillText("注: 该图为结构与信号关系示意, 不代表具体机械尺寸。", 22, canvas.height
1541 - 18);
1542 }
1543
1544 function drawMetricsVisuals() {
1545     const r = state.result;
1546     if (!r) return;
1547     drawMetricsBar("metricsBarCanvas", r.metrics);
1548     drawMetricsRadar("metricsRadarCanvas", r.metrics);
1549 }
1550
1551 function drawMetricsBar(canvasId, metrics) {
1552     const canvas = setupCanvas(canvasId);
1553     const ctx = canvas.getContext("2d");
1554     clearChart(ctx, canvas);
1555     const pad = { l: 54, r: 18, t: 22, b: 74 };
1556     const maxY = Math.max(...metrics.flatMap(m => [m.maxDeg, m.rmsDeg]), 0.1) * 1.15;
1557     drawAxes(ctx, canvas, pad, [0, metrics.length], [0, maxY], { xLabel: "处理方式", yLabel: "
1558 误差 / deg" });
1559     const plotW = canvas.width - pad.l - pad.r;
1560     const groupW = plotW / metrics.length;
1561     const bw = Math.min(24, groupW * 0.24);
1562     metrics.forEach((m, i) => {
1563         const baseX = pad.l + i * groupW + groupW / 2;
1564         const y0 = map(0, 0, maxY, canvas.height - pad.b, pad.t);
1565         const yMax = map(m.maxDeg, 0, maxY, canvas.height - pad.b, pad.t);
1566         const yRms = map(m.rmsDeg, 0, maxY, canvas.height - pad.b, pad.t);
1567         ctx.fillStyle = "#57a9df";
1568         ctx.fillRect(baseX - bw - 2, yMax, bw, y0 - yMax);
1569         ctx.fillStyle = "#d9812b";
1570         ctx.fillRect(baseX + 2, yRms, bw, y0 - yRms);
1571         ctx.save();
1572         ctx.translate(baseX, canvas.height - pad.b + 12);
1573         ctx.rotate(-Math.PI / 5);
1574         ctx.fillStyle = "#60788a";
1575         ctx.font = "12px Microsoft YaHei";
1576         ctx.textAlign = "right";
1577         ctx.fillText(shortMetricName(m.name), 0, 0);
1578         ctx.restore();
1579     });
1580 }
1581
1582 function drawMetricsRadar(canvasId, metrics) {
1583     const canvas = setupCanvas(canvasId);
1584     const ctx = canvas.getContext("2d");
1585     clearChart(ctx, canvas);
1586     const cx = canvas.width / 2;
1587     const cy = canvas.height / 2 + 8;
1588     const radius = Math.min(canvas.width, canvas.height) * 0.33;
1589     const maxRms = Math.max(...metrics.map(m => m.rmsDeg), 1e-6);
1590     const scores = metrics.map(m => Math.max(0.05, 1 - m.rmsDeg / (maxRms * 1.08)));

```

```

1591
1592     ctx.strokeStyle = "#d8eaf5";
1593     ctx.fillStyle = "#f6fbff";
1594     for (let ring = 4; ring >= 1; ring--) {
1595         ctx.beginPath();
1596         for (let i = 0; i < metrics.length; i++) {
1597             const a = -Math.PI / 2 + i * Math.PI * 2 / metrics.length;
1598             const x = cx + Math.cos(a) * radius * ring / 4;
1599             const y = cy + Math.sin(a) * radius * ring / 4;
1600             if (i === 0) ctx.moveTo(x, y); else ctx.lineTo(x, y);
1601         }
1602         ctx.closePath();
1603         if (ring === 4) ctx.fill();
1604         ctx.stroke();
1605     }
1606     ctx.strokeStyle = "#b9d8e9";
1607     metrics.forEach((m, i) => {
1608         const a = -Math.PI / 2 + i * Math.PI * 2 / metrics.length;
1609         ctx.beginPath();
1610         ctx.moveTo(cx, cy);
1611         ctx.lineTo(cx + Math.cos(a) * radius, cy + Math.sin(a) * radius);
1612         ctx.stroke();
1613         ctx.fillStyle = "#60788a";
1614         ctx.font = "12px Microsoft YaHei";
1615         ctx.textAlign = Math.cos(a) > 0.2 ? "left" : (Math.cos(a) < -0.2 ? "right" : "center");
1616         ctx.textBaseline = Math.sin(a) > 0.4 ? "top" : (Math.sin(a) < -0.4 ? "bottom" : "middle");
1617         ctx.fillText(shortMetricName(m.name), cx + Math.cos(a) * (radius + 12), cy + Math.sin(a)
1618 * (radius + 12));
1619     });
1620     ctx.beginPath();
1621     scores.forEach((s, i) => {
1622         const a = -Math.PI / 2 + i * Math.PI * 2 / scores.length;
1623         const x = cx + Math.cos(a) * radius * s;
1624         const y = cy + Math.sin(a) * radius * s;
1625         if (i === 0) ctx.moveTo(x, y); else ctx.lineTo(x, y);
1626     });
1627     ctx.closePath();
1628     ctx.fillStyle = "rgba(22, 114, 173, 0.18)";
1629     ctx.strokeStyle = "#1672ad";
1630     ctx.lineWidth = 2;
1631     ctx.fill();
1632     ctx.stroke();
1633     ctx.fillStyle = "#60788a";
1634     ctx.font = "12px Microsoft YaHei";
1635     ctx.textAlign = "center";
1636     ctx.textBaseline = "top";
1637     ctx.fillText("RMS 误差越小, 顶点越靠外", cx, 12);
1638 }
1639
1640 function drawLines(canvasId, x, series, opt) {
1641     const canvas = setupCanvas(canvasId);
1642     const ctx = canvas.getContext("2d");
1643     const pad = { l: 58, r: 18, t: 18, b: 42 };

```

```
1644     const flatY = series.flatMap(s => s.y);
1645     const xr = range(x);
1646     const yr = paddedRange(flatY);
1647     clearChart(ctx, canvas);
1648     drawAxes(ctx, canvas, pad, xr, yr, opt);
1649     for (const s of series) {
1650         ctx.strokeStyle = s.color;
1651         ctx.lineWidth = 1.7;
1652         ctx.beginPath();
1653         for (let i = 0; i < x.length; i++) {
1654             const px = map(x[i], xr[0], xr[1], pad.l, canvas.width - pad.r);
1655             const py = map(s.y[i], yr[0], yr[1], canvas.height - pad.b, pad.t);
1656             if (i === 0) ctx.moveTo(px, py); else ctx.lineTo(px, py);
1657         }
1658         ctx.stroke();
1659     }
1660 }
1661
1662 function drawXY(canvasId, series, opt) {
1663     const canvas = setupCanvas(canvasId);
1664     const ctx = canvas.getContext("2d");
1665     const pad = { l: 58, r: 18, t: 18, b: 42 };
1666     const xr = paddedRange(series.flatMap(s => s.x));
1667     const yr = paddedRange(series.flatMap(s => s.y));
1668     clearChart(ctx, canvas);
1669     drawAxes(ctx, canvas, pad, xr, yr, opt);
1670     for (const s of series) {
1671         ctx.strokeStyle = s.color;
1672         ctx.lineWidth = s.color === colors.grey ? 1.3 : 1.8;
1673         ctx.beginPath();
1674         for (let i = 0; i < s.x.length; i++) {
1675             const px = map(s.x[i], xr[0], xr[1], pad.l, canvas.width - pad.r);
1676             const py = map(s.y[i], yr[0], yr[1], canvas.height - pad.b, pad.t);
1677             if (i === 0) ctx.moveTo(px, py); else ctx.lineTo(px, py);
1678         }
1679         ctx.stroke();
1680     }
1681 }
1682
1683 function drawBars(canvasId, x, y, opt) {
1684     const canvas = setupCanvas(canvasId);
1685     const ctx = canvas.getContext("2d");
1686     const pad = { l: 58, r: 18, t: 18, b: 42 };
1687     const xr = [0, Math.max(...x) * 1.08];
1688     const yr = paddedRange(y.concat([-80, 0]));
1689     clearChart(ctx, canvas);
1690     drawAxes(ctx, canvas, pad, xr, yr, opt);
1691     const w = Math.max(4, (canvas.width - pad.l - pad.r) / (x.length * 2.8));
1692     ctx.fillStyle = opt.color;
1693     ctx.strokeStyle = opt.color;
1694     for (let i = 0; i < x.length; i++) {
1695         const px = map(x[i], xr[0], xr[1], pad.l, canvas.width - pad.r);
1696         const py = map(y[i], yr[0], yr[1], canvas.height - pad.b, pad.t);
```



```
1697     const y0 = map(yr[0], yr[0], yr[1], canvas.height - pad.b, pad.t);
1698     ctx.fillRect(px - w / 2, py, w, Math.max(1, y0 - py));
1699     ctx.fillStyle = colors.muted || "#60788a";
1700     ctx.font = "12px Microsoft YaHei";
1701     ctx.textAlign = "center";
1702     ctx.fillText(`${opt.labels[i]}x`, px, py - 5);
1703     ctx.fillStyle = opt.color;
1704   }
1705 }
1706
1707 function drawGridFloor(ctx, canvas, y) {
1708   ctx.strokeStyle = "rgba(141, 168, 185, 0.22)";
1709   ctx.lineWidth = 1;
1710   for (let x = -canvas.width; x < canvas.width * 1.5; x += 34) {
1711     ctx.beginPath();
1712     ctx.moveTo(x, y);
1713     ctx.lineTo(x + 220, canvas.height);
1714     ctx.stroke();
1715   }
1716   for (let i = 0; i < 7; i++) {
1717     const yy = y + i * 22;
1718     ctx.beginPath();
1719     ctx.moveTo(0, yy);
1720     ctx.lineTo(canvas.width, yy);
1721     ctx.stroke();
1722   }
1723 }
1724
1725 function drawRingSector(ctx, cx, cy, rxO, ryO, rxI, ryI, a0, a1, fill) {
1726   ctx.beginPath();
1727   for (let i = 0; i <= 16; i++) {
1728     const a = a0 + (a1 - a0) * i / 16;
1729     const x = cx + rxO * Math.cos(a);
1730     const y = cy + ryO * Math.sin(a);
1731     if (i === 0) ctx.moveTo(x, y); else ctx.lineTo(x, y);
1732   }
1733   for (let i = 16; i >= 0; i--) {
1734     const a = a0 + (a1 - a0) * i / 16;
1735     ctx.lineTo(cx + rxI * Math.cos(a), cy + ryI * Math.sin(a));
1736   }
1737   ctx.closePath();
1738   ctx.fillStyle = fill;
1739   ctx.fill();
1740   ctx.strokeStyle = "rgba(255,255,255,0.65)";
1741   ctx.lineWidth = 1;
1742   ctx.stroke();
1743 }
1744
1745 function ellipsePoint(cx, cy, rx, ry, angle) {
1746   return { x: cx + rx * Math.cos(angle), y: cy + ry * Math.sin(angle) };
1747 }
1748
1749 function drawHallChip(ctx, x, y, label, color) {
```

```
1750     roundRect(ctx, x, y, 72, 34, 7);
1751     ctx.fillStyle = "#ffffff";
1752     ctx.fill();
1753     ctx.strokeStyle = color;
1754     ctx.lineWidth = 2;
1755     ctx.stroke();
1756     ctx.fillStyle = color;
1757     ctx.font = "700 12px Microsoft YaHei";
1758     ctx.textAlign = "center";
1759     ctx.textBaseline = "middle";
1760     ctx.fillText(label, x + 36, y + 17);
1761 }
1762
1763 function drawArrow(ctx, x1, y1, x2, y2, color) {
1764     const a = Math.atan2(y2 - y1, x2 - x1);
1765     ctx.strokeStyle = color;
1766     ctx.fillStyle = color;
1767     ctx.lineWidth = 2;
1768     ctx.beginPath();
1769     ctx.moveTo(x1, y1);
1770     ctx.lineTo(x2, y2);
1771     ctx.stroke();
1772     ctx.beginPath();
1773     ctx.moveTo(x2, y2);
1774     ctx.lineTo(x2 - 9 * Math.cos(a - 0.45), y2 - 9 * Math.sin(a - 0.45));
1775     ctx.lineTo(x2 - 9 * Math.cos(a + 0.45), y2 - 9 * Math.sin(a + 0.45));
1776     ctx.closePath();
1777     ctx.fill();
1778 }
1779
1780 function drawAngleArc(ctx, cx, cy, rx, ry, a0, a1, color, label) {
1781     ctx.strokeStyle = color;
1782     ctx.fillStyle = color;
1783     ctx.lineWidth = 2;
1784     ctx.beginPath();
1785     for (let i = 0; i <= 32; i++) {
1786         const a = a0 + (a1 - a0) * i / 32;
1787         const x = cx + rx * Math.cos(a);
1788         const y = cy + ry * Math.sin(a);
1789         if (i === 0) ctx.moveTo(x, y); else ctx.lineTo(x, y);
1790     }
1791     ctx.stroke();
1792     const am = (a0 + a1) / 2;
1793     ctx.font = "700 12px Microsoft YaHei";
1794     ctx.textAlign = "center";
1795     ctx.textBaseline = "middle";
1796     ctx.fillText(label, cx + (rx + 22) * Math.cos(am), cy + (ry + 22) * Math.sin(am));
1797 }
1798
1799 function drawInfoCard(ctx, x, y, w, h, rows) {
1800     roundRect(ctx, x, y, w, h, 8);
1801     ctx.fillStyle = "rgba(255,255,255,0.88)";
1802     ctx.fill();
```

```
1803     ctx.strokeStyle = "#bde0f5";
1804     ctx.lineWidth = 1.5;
1805     ctx.stroke();
1806     ctx.font = "12px Microsoft YaHei";
1807     rows.forEach((row, i) => {
1808         const yy = y + 18 + i * 24;
1809         ctx.fillStyle = "#60788a";
1810         ctx.textAlign = "left";
1811         ctx.fillText(row[0], x + 14, yy);
1812         ctx.fillStyle = "#19384d";
1813         ctx.font = "700 12px Microsoft YaHei";
1814         ctx.textAlign = "right";
1815         ctx.fillText(row[1], x + w - 14, yy);
1816         ctx.font = "12px Microsoft YaHei";
1817     });
1818 }
1819
1820 function roundRect(ctx, x, y, w, h, r) {
1821     const rr = Math.min(r, w / 2, h / 2);
1822     ctx.beginPath();
1823     ctx.moveTo(x + rr, y);
1824     ctx.lineTo(x + w - rr, y);
1825     ctx.quadraticCurveTo(x + w, y, x + w, y + rr);
1826     ctx.lineTo(x + w, y + h - rr);
1827     ctx.quadraticCurveTo(x + w, y + h, x + w - rr, y + h);
1828     ctx.lineTo(x + rr, y + h);
1829     ctx.quadraticCurveTo(x, y + h, x, y + h - rr);
1830     ctx.lineTo(x, y + rr);
1831     ctx.quadraticCurveTo(x, y, x + rr, y);
1832     ctx.closePath();
1833 }
1834
1835 function shortMetricName(name) {
1836     return name
1837         .replace("原始 atan2", "原始")
1838         .replace("零偏/幅值/相位", "基础")
1839         .replace("谐波补偿", "谐波")
1840         .replace("角度 LPF", "LPF")
1841         .replace("alpha-beta", "α-β")
1842         .replace("离散 ESO", "ESO");
1843 }
1844
1845 function setupCanvas(id) {
1846     const canvas = $(id);
1847     const rect = canvas.getBoundingClientRect();
1848     canvas.width = Math.max(320, Math.floor(rect.width));
1849     canvas.height = Math.max(220, Math.floor(rect.height));
1850     return canvas;
1851 }
1852
1853 function clearChart(ctx, canvas) {
1854     ctx.clearRect(0, 0, canvas.width, canvas.height);
1855     ctx.fillStyle = "#fff";
```

```
1856     ctx.fillRect(0, 0, canvas.width, canvas.height);
1857 }
1858
1859 function drawAxes(ctx, canvas, pad, xr, yr, opt) {
1860     ctx.strokeStyle = colors.grid;
1861     ctx.lineWidth = 1;
1862     ctx.font = "12px Microsoft YaHei";
1863     ctx.fillStyle = "#60788a";
1864     ctx.textAlign = "right";
1865     ctx.textBaseline = "middle";
1866     for (let i = 0; i <= 5; i++) {
1867         const yv = yr[0] + (yr[1] - yr[0]) * i / 5;
1868         const py = map(yv, yr[0], yr[1], canvas.height - pad.b, pad.t);
1869         ctx.beginPath();
1870         ctx.moveTo(pad.l, py);
1871         ctx.lineTo(canvas.width - pad.r, py);
1872         ctx.stroke();
1873         ctx.fillText(shortNum(yv), pad.l - 7, py);
1874     }
1875     ctx.textAlign = "center";
1876     ctx.textBaseline = "top";
1877     for (let i = 0; i <= 5; i++) {
1878         const xv = xr[0] + (xr[1] - xr[0]) * i / 5;
1879         const px = map(xv, xr[0], xr[1], pad.l, canvas.width - pad.r);
1880         ctx.beginPath();
1881         ctx.moveTo(px, pad.t);
1882         ctx.lineTo(px, canvas.height - pad.b);
1883         ctx.stroke();
1884         ctx.fillText(shortNum(xv), px, canvas.height - pad.b + 7);
1885     }
1886     ctx.strokeStyle = colors.axis;
1887     ctx.strokeRect(pad.l, pad.t, canvas.width - pad.l - pad.r, canvas.height - pad.t - pad.b);
1888     ctx.fillStyle = "#19384d";
1889     ctx.textAlign = "center";
1890     ctx.fillText(opt.xLabel || "", (pad.l + canvas.width - pad.r) / 2, canvas.height - 19);
1891     ctx.save();
1892     ctx.translate(16, (pad.t + canvas.height - pad.b) / 2);
1893     ctx.rotate(-Math.PI / 2);
1894     ctx.fillText(opt.yLabel || "", 0, 0);
1895     ctx.restore();
1896 }
1897
1898 function exportDashboardPng() {
1899     const canvases = ["modelCanvas", "voltageCanvas", "lissajousCanvas", "fftCanvas",
1900 "errorCanvas"].map(id => $(id));
1901     const w = 1500, h = 1320;
1902     const out = document.createElement("canvas");
1903     out.width = w;
1904     out.height = h;
1905     const ctx = out.getContext("2d");
1906     ctx.fillStyle = "#ffffff";
1907     ctx.fillRect(0, 0, w, h);
1908     ctx.fillStyle = "#19384d";
```

```

1909     ctx.font = "700 28px Microsoft YaHei";
1910     ctx.fillText("双线性 Hall 高精度位置检测与补偿仿真结果", 40, 50);
1911     ctx.font = "16px Microsoft YaHei";
1912     ctx.fillStyle = "#60788a";
1913     ctx.fillText(`工况 : ${scenarioName(state.params.scenario)}      ESO 带宽 :
1914 ${state.params.esoHz} Hz   采样: ${state.params.sampleRate} Hz`, 40, 82);
1915     const boxes = [
1916         [40, 120, 1420, 330],
1917         [40, 500, 680, 350], [780, 500, 680, 350],
1918         [40, 910, 680, 350], [780, 910, 680, 350]
1919     ];
1920     canvases.forEach((c, i) => {
1921         ctx.drawImage(c, boxes[i][0], boxes[i][1], boxes[i][2], boxes[i][3]);
1922     });
1923     download(out.toDataURL("image/png"), "dual_hall_detection_dashboard.png");
1924     toast("已导出 PNG");
1925 }
1926
1927 function exportCsv() {
1928     const rows = [["method", "max_error_deg", "rms_error_deg", "note"]];
1929     for (const m of state.result.metrics) rows.push([m.name, m.maxDeg.toFixed(8),
1930 m.rmsDeg.toFixed(8), m.note]);
1931     downloadText(rows.map(r => r.map(csvCell).join(",")).join("\n"),
1932 "dual_hall_detection_metrics.csv", "text/csv;charset=utf-8");
1933     toast("已导出 CSV");
1934 }
1935
1936 function exportParams() {
1937     downloadText(JSON.stringify(state.params, null, 2), "dual_hall_detection_params.json",
1938 "application/json;charset=utf-8");
1939     toast("已导出参数 JSON");
1940 }
1941
1942 function rowMetric(name, err, note) {
1943     const deg = err.map(rad2deg);
1944     return { name, maxDeg: maxAbs(deg), rmsDeg: rms(deg), note };
1945 }
1946
1947 function firstOrderLpf(x, alpha) {
1948     const y = new Array(x.length);
1949     y[0] = x[0];
1950     for (let i = 1; i < x.length; i++) y[i] = alpha * y[i - 1] + (1 - alpha) * x[i];
1951     return y;
1952 }
1953
1954 function lpfAlpha(ts, cutoffHz) {
1955     const tau = 1 / (2 * Math.PI * cutoffHz);
1956     return Math.exp(-ts / tau);
1957 }
1958
1959 function angleError(thetaEst, thetaMag, polePairs) {
1960     return thetaEst.map((v, i) => wrapPi(v - thetaMag[i]) / polePairs);
1961 }

```

```

1962
1963 function basisMatrix(theta, orders) {
1964     return theta.map(th => {
1965         const row = [1];
1966         for (const n of orders) {
1967             row.push(Math.sin(n * th), Math.cos(n * th));
1968         }
1969         return row;
1970     });
1971 }
1972
1973 function leastSquares(A, b) {
1974     const m = A.length;
1975     const n = A[0].length;
1976     const ata = Array.from({ length: n }, () => new Array(n).fill(0));
1977     const atb = new Array(n).fill(0);
1978     for (let i = 0; i < m; i++) {
1979         for (let c = 0; c < n; c++) {
1980             atb[c] += A[i][c] * b[i];
1981             for (let d = 0; d < n; d++) ata[c][d] += A[i][c] * A[i][d];
1982         }
1983     }
1984     for (let i = 0; i < n; i++) ata[i][i] += 1e-8;
1985     return solveLinear(ata, atb);
1986 }
1987
1988 function solveLinear(A, b) {
1989     const n = b.length;
1990     const M = A.map((row, i) => row.concat(b[i]));
1991     for (let k = 0; k < n; k++) {
1992         let maxRow = k;
1993         for (let i = k + 1; i < n; i++) if (Math.abs(M[i][k]) > Math.abs(M[maxRow][k])) maxRow
1994 = i;
1995         [M[k], M[maxRow]] = [M[maxRow], M[k]];
1996         const pivot = Math.abs(M[k][k]) < 1e-12 ? 1e-12 : M[k][k];
1997         for (let j = k; j <= n; j++) M[k][j] /= pivot;
1998         for (let i = 0; i < n; i++) {
1999             if (i === k) continue;
2000             const factor = M[i][k];
2001             for (let j = k; j <= n; j++) M[i][j] -= factor * M[k][j];
2002         }
2003     }
2004     return M.map(row => row[n]);
2005 }
2006
2007 function multiply(A, x) {
2008     return A.map(row => row.reduce((sum, v, i) => sum + v * x[i], 0));
2009 }
2010
2011 function adcQuantize(v, p) {
2012     const vmax = Math.pow(2, p.adcBits) - 1;
2013     const clipped = clamp(v, 0, p.vref);
2014     return Math.round(clipped / p.vref * vmax) / vmax * p.vref;

```

```
2015     }
2016
2017     function unwrap(arr) {
2018         const out = new Array(arr.length);
2019         out[0] = arr[0];
2020         let offset = 0;
2021         for (let i = 1; i < arr.length; i++) {
2022             let d = arr[i] - arr[i - 1];
2023             if (d > Math.PI) offset -= 2 * Math.PI;
2024             if (d < -Math.PI) offset += 2 * Math.PI;
2025             out[i] = arr[i] + offset;
2026         }
2027         return out;
2028     }
2029
2030     function alignAngle(theta, ref) {
2031         const offset = Math.round((theta[0] - ref[0]) / (2 * Math.PI)) * 2 * Math.PI;
2032         return theta.map(v => v - offset);
2033     }
2034
2035     function wrapPi(x) {
2036         return ((x + Math.PI) % (2 * Math.PI) + 2 * Math.PI) % (2 * Math.PI) - Math.PI;
2037     }
2038
2039     function smoothStep(x) {
2040         const u = clamp(x, 0, 1);
2041         return u * u * (3 - 2 * u);
2042     }
2043
2044     function mulberry32(seed) {
2045         let a = seed >>> 0;
2046         return function() {
2047             a += 0x6D2B79F5;
2048             let t = a;
2049             t = Math.imul(t ^ t >>> 15, t | 1);
2050             t ^= t + Math.imul(t ^ t >>> 7, t | 61);
2051             return ((t ^ t >>> 14) >>> 0) / 4294967296;
2052         };
2053     }
2054
2055     function randn(rng) {
2056         const u = Math.max(rng(), 1e-12);
2057         const v = Math.max(rng(), 1e-12);
2058         return Math.sqrt(-2 * Math.log(u)) * Math.cos(2 * Math.PI * v);
2059     }
2060
2061     function map(v, a, b, c, d) {
2062         return c + (v - a) / Math.max(b - a, 1e-12) * (d - c);
2063     }
2064
2065     function range(arr) {
2066         return [Math.min(...arr), Math.max(...arr)];
2067     }
```

```
2068
2069 function paddedRange(arr) {
2070   let lo = Math.min(...arr);
2071   let hi = Math.max(...arr);
2072   if (Math.abs(hi - lo) < 1e-9) { hi += 1; lo -= 1; }
2073   const pad = 0.08 * (hi - lo);
2074   return [lo - pad, hi + pad];
2075 }
2076
2077 function mean(arr) {
2078   return arr.reduce((s, v) => s + v, 0) / arr.length;
2079 }
2080
2081 function rms(arr) {
2082   return Math.sqrt(mean(arr.map(v => v * v)));
2083 }
2084
2085 function min(arr) { return Math.min(...arr); }
2086 function max(arr) { return Math.max(...arr); }
2087 function maxAbs(arr) { return Math.max(...arr.map(v => Math.abs(v))); }
2088 function deg2rad(v) { return v * Math.PI / 180; }
2089 function rad2deg(v) { return v * 180 / Math.PI; }
2090 function clamp(v, lo, hi) { return Math.min(Math.max(v, lo), hi); }
2091 function fmtDeg(v) { return `${v.toFixed(4)}°`; }
2092 function shortNum(v) {
2093   const av = Math.abs(v);
2094   if (av >= 100) return v.toFixed(0);
2095   if (av >= 10) return v.toFixed(1);
2096   if (av >= 1) return v.toFixed(2);
2097   return v.toFixed(3);
2098 }
2099
2100 function scenarioName(v) {
2101   return ({ constant: "恒速", variable: "变速", impact: "冲击", load: "负载扰动" })[v] || v;
2102 }
2103
2104 function csvCell(v) {
2105   const s = String(v).replace(/"/g, "");
2106   return `"${s}"`;
2107 }
2108
2109 function downloadText(text, filename, mime) {
2110   const blob = new Blob(["\u0000", text], { type: mime });
2111   const url = URL.createObjectURL(blob);
2112   download(url, filename);
2113   setTimeout(() => URL.revokeObjectURL(url), 500);
2114 }
2115
2116 function download(url, filename) {
2117   const a = document.createElement("a");
2118   a.href = url;
2119   a.download = filename;
2120   document.body.appendChild(a);
```



```
2121     a.click();
2122     a.remove();
2123 }
2124
2125 function toast(text) {
2126     const el = $("toast");
2127     el.textContent = text;
2128     el.classList.add("show");
2129     clearTimeout(toast._timer);
2130     toast._timer = setTimeout(() => el.classList.remove("show"), 1800);
2131 }
2132
2133 function resetDefaults() {
2134     const defaults = {
2135         polePairs: 6, speedRpm: 100, simTime: 0.5, sampleRate: 5000, adcBits: 16, vref: 3.3,
2136         mountAngleDeg: 90,
2137         enableOffset: true, enableAmp: true, enablePhase: true, enableHarm: true, enableNoise:
2138         true, enableAdc: true,
2139         offsetS: 0.05, offsetC: -0.08, ampS: 1.00, ampC: 0.85, phaseDeg: 5, noiseRms: 0.010,
2140         harmScale: 1, harmScaleValue: 1, lpfHz: 300, abAlpha: 0.22, abBeta: 0.018, esoHz: 60,
2141         signalOrders: "2,3,5,6", angleOrders: "1,2,3,4,5,6,7,8",
2142         scenario: "constant", speedRatio: 1.8, impactDeg: 2, loadDrop: 0.25, seed: 1808
2143     };
2144     for (const [key, value] of Object.entries(defaults)) {
2145         const el = $(key);
2146         if (!el) continue;
2147         if (el.type === "checkbox") el.checked = value;
2148         else el.value = value;
2149     }
2150     syncAllSliders();
2151     simulate();
2152     toast("已恢复默认参数");
2153 }
2154
2155 const sliderPairs = [
2156     ["speedRpmSlider", "speedRpm"],
2157     ["mountAngleDegSlider", "mountAngleDeg"],
2158     ["phaseDegSlider", "phaseDeg"],
2159     ["noiseRmsSlider", "noiseRms"],
2160     ["harmScale", "harmScaleValue"],
2161     ["lpfHzSlider", "lpfHz"],
2162     ["abAlphaSlider", "abAlpha"],
2163     ["abBetaSlider", "abBeta"],
2164     ["esoHzSlider", "esoHz"],
2165     ["speedRatioSlider", "speedRatio"],
2166     ["impactDegSlider", "impactDeg"],
2167     ["loadDropSlider", "loadDrop"]
2168 ];
2169
2170 function setupSyncedSliders() {
2171     sliderPairs.forEach(([sliderId, valueId]) => {
2172         const slider = $(sliderId);
2173         const value = $(valueId);
```

```
2174     if (!slider || !value) return;
2175     slider.addEventListener("input", () => {
2176         value.value = slider.value;
2177         simulate();
2178     });
2179     value.addEventListener("input", () => {
2180         slider.value = value.value;
2181         simulate();
2182     });
2183 });
2184 syncAllSliders();
2185 }
2186
2187 function syncAllSliders() {
2188     sliderPairs.forEach(([sliderId, valueId]) => {
2189         const slider = $(sliderId);
2190         const value = $(valueId);
2191         if (slider && value) slider.value = value.value;
2192     });
2193 }
2194
2195 function bindEvents() {
2196     $("runBtn").addEventListener("click", simulate);
2197     $("resetBtn").addEventListener("click", resetDefaults);
2198     $("exportPngBtn").addEventListener("click", exportDashboardPng);
2199     $("exportCsvBtn").addEventListener("click", exportCsv);
2200     $("exportParamBtn").addEventListener("click", exportParams);
2201     $("reportPngBtn").addEventListener("click", exportDashboardPng);
2202     $("reportCsvBtn").addEventListener("click", exportCsv);
2203     $("reportParamBtn").addEventListener("click", exportParams);
2204     setupSyncedSliders();
2205     for (const id of ids) {
2206         const el = $(id);
2207         if (!el) continue;
2208         el.addEventListener("change", simulate);
2209     }
2210     document.querySelectorAll(".tab").forEach(btn => {
2211         btn.addEventListener("click", () => {
2212             document.querySelectorAll(".tab").forEach(v => v.classList.remove("active"));
2213             document.querySelectorAll(".view").forEach(v => v.classList.remove("active"));
2214             btn.classList.add("active");
2215             $(btn.dataset.view).classList.add("active");
2216             renderAll();
2217         });
2218     });
2219     window.addEventListener("resize", () => {
2220         clearTimeout(bindEvents._timer);
2221         bindEvents._timer = setTimeout(renderAll, 120);
2222     });
2223     startModelAnimation();
2224 }
2225
2226 function startModelAnimation() {
```

```
2227     if (state.modelAnimReq) cancelAnimationFrame(state.modelAnimReq);
2228     state.modelLastTs = 0;
2229     const tick = (ts) => {
2230         if (!state.modelLastTs) state.modelLastTs = ts;
2231         const dt = Math.min(0.05, (ts - state.modelLastTs) / 1000);
2232         state.modelLastTs = ts;
2233         if (state.result) {
2234             const rpm = state.result.p.speedRpm || 0;
2235             const visualRps = clamp(rpm / 60, 0.05, 2.4);
2236             state.modelTheta = wrapPi(state.modelTheta + visualRps * 2 * Math.PI * dt);
2237             if ($("dashboard").classList.contains("active")) drawMotorModel();
2238         }
2239         state.modelAnimReq = requestAnimationFrame(tick);
2240     };
2241     state.modelAnimReq = requestAnimationFrame(tick);
2242 }
2243
2244     bindEvents();
2245     simulate();
2246 </script>
2247 </body>
2248 </html>
```